Natural language processing

Text to query

Finn Årup Nielsen
DTU Compute, Technical University of Denmark

March 19, 2026

Text to query

From a user question generate a database query that can retrieve data.

Integration of LLM and knowledge graphs (graph-RAG)

"What is the width of a Yamaha P-150?"

The text-to-query system generates a SPARQL query

```
SELECT ?value WHERE { kb:Q1 kbt:P2 ?value . }
```

which when executed on a SPARQL endpoint (*Keyboards*) gives

```
1385
```

This information comes from a page on the *Keyboards* Wikibase at https://keyboards.wikibase.cloud/wiki/Item:Q1 which have been converted to a knowledge graph.

Integration with knowledge graphs could make multi-hop retrieval.

Retrieval-augmented generation stage

RAG 1.0

- Retrieve top-k documents/chunks
- Add them to the prompt

RAG 2.0

- query transformation
- hybrid search
- reranking of retrieval documents

RAG 3.0

- Graph retrieval ← We are here today!
- Iterative reasoning ← and possibly also here!
- ...

Setting up a knowledge graph

Here: *Keyboards* from a Wikibase that I have set up with technical information about musical keyboards at https://keyboards.wikibase.cloud/

I have extracted all the triple available in the triple store associate with the *Keyboards* wikibase with a `CONSTRUCT` SPARQL query.

```
from rdflib import Graph

endpoint = "https://keyboards.wikibase.cloud/query/sparql"
query = """
CONSTRUCT { ?s ?p ?o }
WHERE { ?s ?p ?o . }
"""
...
response = requests.get(endpoint, params={"query": query}, headers=headers)
g = Graph()
g.parse(data=response.text, format="nt")
g.serialize("keyboards.nt", format="nt")
```

The `keyboards.nt` file is made available.

QLever

QLever is a triple store (graph database) developed from University of Freiburg with SPARQL search

Installation: `pip install qlever`

To set up and instance with data you need to define a `Qleverfile` (lowercase "l")

After the `Qleverfile` is defined then do

```
qlever index
qlever start
```

to index the data and start serving.

A docker container is running (see with `docker ps`) and a SPARQL endpoint should now run on http://localhost:7070.

Qleverfile

```
[data]
NAME = keyboards
DESCRIPTION = Keyboards Wikibase snapshot

[index]
INPUT_FILES = keyboards.nt
CAT_INPUT_FILES = cat keyboards.nt

[server]
PORT = 7070
TIMEOUT = 30s

[runtime]
SYSTEM = docker
IMAGE = docker.io/adfreiburg/qlever:latest

[ui]
UI_CONFIG = default
UI_PORT = 8176
UI_SYSTEM = docker
UI_IMAGE = docker.io/adfreiburg/qlever-ui
```

QLever UI

A user interface can be started with

```
qlever ui
```

A SPARQL UI for *Keyboards* is now available at http://localhost:8176.

A SPARQL query such as

```
DESCRIBE <https://keyboards.wikibase.cloud/entity/Q1>
```

should return something in the “Query results”.

RDFLib

Instead of QLever a combination of RDFLib and FastAPI can also work as a SPARQL endpoint, e.g., with this simplified interface.

```
from fastapi import FastAPI, Request, Response
from rdflib import Graph
import asyncio

app = FastAPI()  # The Web service

g = Graph()  # The knowledge graph
g.parse("keyboards.nt")  # Load RDF data

@app.api_route("/sparql", methods=["GET", "POST"])
async def sparql_endpoint(request: Request):
    query = request.query_params.get("query")
    result = await asyncio.to_thread(g.query, query)
    return Response(
            content=result.serialize(format="json"),
            media_type="application/sparql-results+json"
    )
```

Entity recognizing

“What is the width of a Yamaha P-150?”

Possible substrings that denote an item: “Yamaha P-150”, “Yamaha”, “P-150”.

Possible substrings that denote a property: “width”, “width of”.

Use an LLM (CampusAI) to extract these substrings.

Prompt: “For a text-to-SPARQL system I need to look up relevant entities (property and item) URIs. For a question, extract the phrases/words that I can use to query an API to resolve the phrases/words to URIs. The list should be extended with possible variations that the entity labels found in the system I query. The result should be in JSON in a dictionary (fields: properties and items) of list of strings order so the most likely is first in the list. For examples for ”What is the width of a Yamaha P-150”, the result should be”

Entity linking items

Get from “Yamaha P-150” to Q1

```python
def lookup_item(text: str) -> str:
    sparql = f"""
    PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
    PREFIX skos: <http://www.w3.org/2004/02/skos/core#>

    SELECT ?item {{
       ?item rdfs:label | skos:altLabel "{text}"@en
    }}
    """
    results = run_sparql(sparql)
    for result in results:
        return get_qid(result.get('item'))
    return ""
```

Entity linking properties

Get from “width” to P2

```python
def lookup_property(text: str) -> str:
    sparql = f"""
    PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
    PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
    PREFIX wikibase: <http://wikiba.se/ontology#>

    SELECT ?property {{
        ?property_item rdfs:label | skos:altLabel "{text}"@en ;
                       wikibase:directClaim ?property .
    }}"""
    results = run_sparql(sparql)
    for result in results:
        return get_qid(result.get('property'))
    return ""
```

Query construction via templates

The template approach works for questions like “What is the weight of P-150” and “What is the width of Roland SH-101” where the entity linked QID and PID are interpolated into a SPARQL template

```
entities = extract_entities(q.text)
qid = lookup_item(entities.items[0])
pid = lookup_property(entities.properties[0])

sparql = f"""
PREFIX kb: <https://keyboards.wikibase.cloud/entity/>
PREFIX kbt: <https://keyboards.wikibase.cloud/prop/direct/>

SELECT ?value WHERE {{
  kb:{qid} kbt:{pid} ?value .
}}
"""

results = run_sparql(sparql)
```

Query the SPARQL endpoint

Querying the SPARQL endpoint from Python

```python
SPARQL_ENDPOINT = "http://localhost:7070/sparql"

def run_sparql(query: str) -> list[dict]:
    with httpx.Client(timeout=20) as client:
        r = client.get(SPARQL_ENDPOINT, params={"query": query, "format": "json"})
        r.raise_for_status()
        bindings = r.json()["results"]["bindings"]
        result = simplify_bindings(bindings)  # Optional reformatting
        return result
```

Complex query

"What is the smallest Swedish keyboard?"

should/could be converted to the following SPARQL query

```
SELECT ?keyboard WHERE {
  ?manufacturer kbt:P19 kb:Q26 .
  ?keyboard kbt:P6 ?manufacturer .
  ?keyboard kbt:P2 ?width ;
            kbt:P3 ?depth ;
            kbt:P4 ?height .
  BIND((?width * ?depth * ?height) AS ?volume)
}
ORDER BY ?volume
LIMIT 1
```

There is not clear single property for *smallest* and what is meant by "Swedish keyboard"?

Need to discover "Swedish" → Sweden → country of manufactorer → Nord, (...?) → list of keyboards → dimension of these keyboards → ...

Advanced: LLM in a ReAct loop

ReAct: LLM + function calls (e.g., external to APIs) in a loop.

Function calls for text-to-query would be: entity extraction, item lookup, property lookup, SPARQL execution, item representation.

Few-shot prompting: Add example queries to the LLM prompt.

Dynamic few-shot prompting: Add similar example queries to the LLM prompt. Here you need an information retrieval system for the query

Advanced: LLM in a ReAct loop

Example DSPy ReAct agent

```
react_agent = dspy.ReAct(
    SystemQuery,
    tools=[
        execute_sparql,    # runs arbitrary SPARQL against QLever
        get_example_sparql_queries,  # Few-shot examples
        is_valid_sparql,   # optional syntax checker
        search_items,      # label → kb:Qxx
        search_properties, # label → kbt:Pxx
        list_all_properties,  # browse properties
    ],
    max_iters=25,        # how many Thought/Action steps it may take
)
```

ReAct alternates between "Reasoning and Acting". Reasoning is the LLM processing and Acting is calling (external) functions, here, e.g., `list_all_properties`.

Advanced: Set up LLM for DSPy

LLM configuration for DSPy

```
load_dotenv(os.path.expanduser("~/.env"))
api_key = os.getenv("CAMPUSAI_API_KEY")
model = os.getenv("CAMPUSAI_MODEL")
api_url = os.getenv("CAMPUSAI_API_URL")

lm = dspy.LM(
    'openai/' + model,
    api_base=api_url,
    api_key=api_key,
    temperature=0,
    max_tokens=2048,
)
dspy.configure(lm=lm)
```

Advanced: Prompt for DSPy ReAct

```python
class SystemQuery(dspy.Signature):
    """
    Answer the user's question using the keyboards knowledge base.

    You MUST:
    - Translate the natural-language question into a SPARQL query
      that uses the kb:/kbt:/rdfs: vocabulary.
    - First extract relevant phrases that you need to look up
    - The search for relevant items and properties.
    - Call `search_items` to search for an item entity URI (example: `kb:Q45`)
    - Call `search_properties` to search for a property entity URI (example: `kbt:P11`)
    - Be aware that the properties does not correspond to Wikidata's properties.
    - Be aware of the properties are prefixed with `kbt`, e.g., `kbt:P7`
    - If you fail to identify relevant property(ies) list all properties
    - Call `list_all_properties` to list all properties.
    ... (and more)
    """
    question: str = dspy.InputField(desc="User question in natural language about keyboards or related entities.")
    answer: str = dspy.OutputField(desc="Final factual answer to the user's question, in natural language.")
```

Advanced: lookup of properties

SPARQL query for getting properties and their label

```
PREFIX kb: <https://keyboards.wikibase.cloud/entity/>
PREFIX kbt: <https://keyboards.wikibase.cloud/prop/direct/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
PREFIX wikibase: <http://wikiba.se/ontology#>

SELECT DISTINCT (SUBSTR(STR(?item), 41) AS ?id) ?label WHERE {
  ?item wikibase:statements [] ;
        rdfs:label | skos:altLabel ?label .
}
```

Used in `list_all_properties` tool.

Would probably only work for small datasets.

Advanced: Example SPARQL checker

A function for a DSPy ReAct tool that checks locally whether a SPARQL query is valid with RDFLib before submitting it to an (external) SPARQL endpoint.

```python
from rdflib.plugins.sparql.parser import parseQuery

def is_valid_sparql(query: str) -> dict:
    """
    Check whether the given string is valid SPARQL syntax.
    """
    try:
        # Parse to AST
        ast = parseQuery(q)
        return {"valid_sparql": True}
    except Exception as e:
        return {"valid_sparql": False, "error_message": str(e)}
```

Only simple errors are checked.

Few shot examples

The *Keyboards* knowledge graph has SPARQL query examples.

You can get the examples with

```
PREFIX kb: <https://keyboards.wikibase.cloud/entity/>
PREFIX kbt: <https://keyboards.wikibase.cloud/prop/direct/>

SELECT
  ?question ?sparql
WHERE {
  [] kbt:P11 kb:Q43 ;
     kbt:P27 ?question ;
     kbt:P31 ?sparql .
}
```

There are both English and Danish natural language questions. There may be multiple natural language questions for each SPARQL query

"Which keyboards are missing the specification of power consumption?"
→ `SELECT ?keyboard WHERE { ?keyboard kbt:P11 kb:Q14 . FILTER NOT EXISTS { ?keyboard kbt:P16 ?power_consumption . }`

Text-to-query observations

A LLM-only system can only to limited extent generate accurate SPARQL queries.

Relevant few-shots examples improve performances.

For Wikidata/Wikibase-like URIs like Q1234, entity linking is very important. Without it, performance might be very poor.

An iterative ReAct-like approach with a good number of actions performs well, cf. GRASP and SPINACH systems.

Better/larger LLM produces better text-to-query results, e.g., with F1 score 58 (Gemini 2.0 Flash) $\rightarrow$ 82 (o4-mini) reported for GRASP on QALD-10